Technical Report

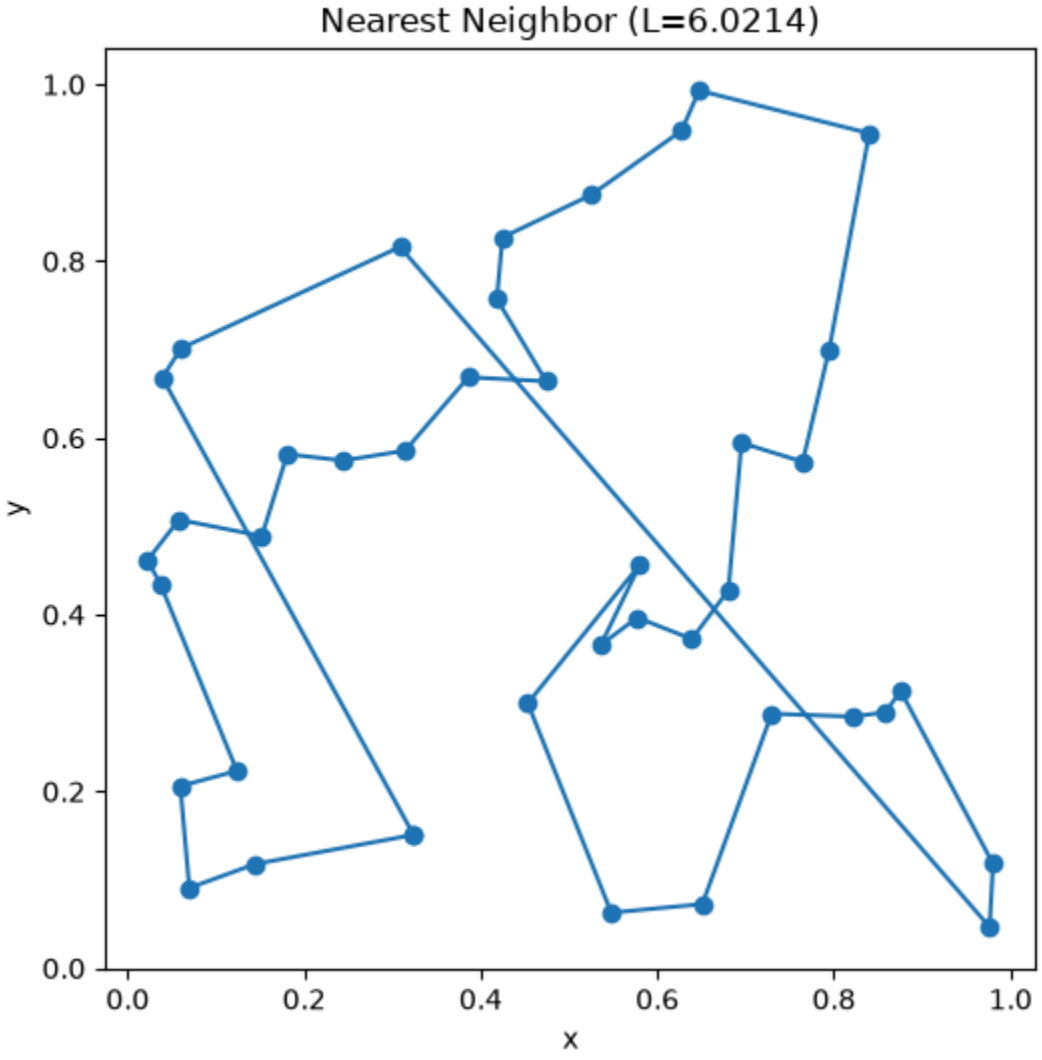
Title: Traveling Salesperson Problem using SA & GA Algorithms

Author: Abraham Simon
9th October 2026

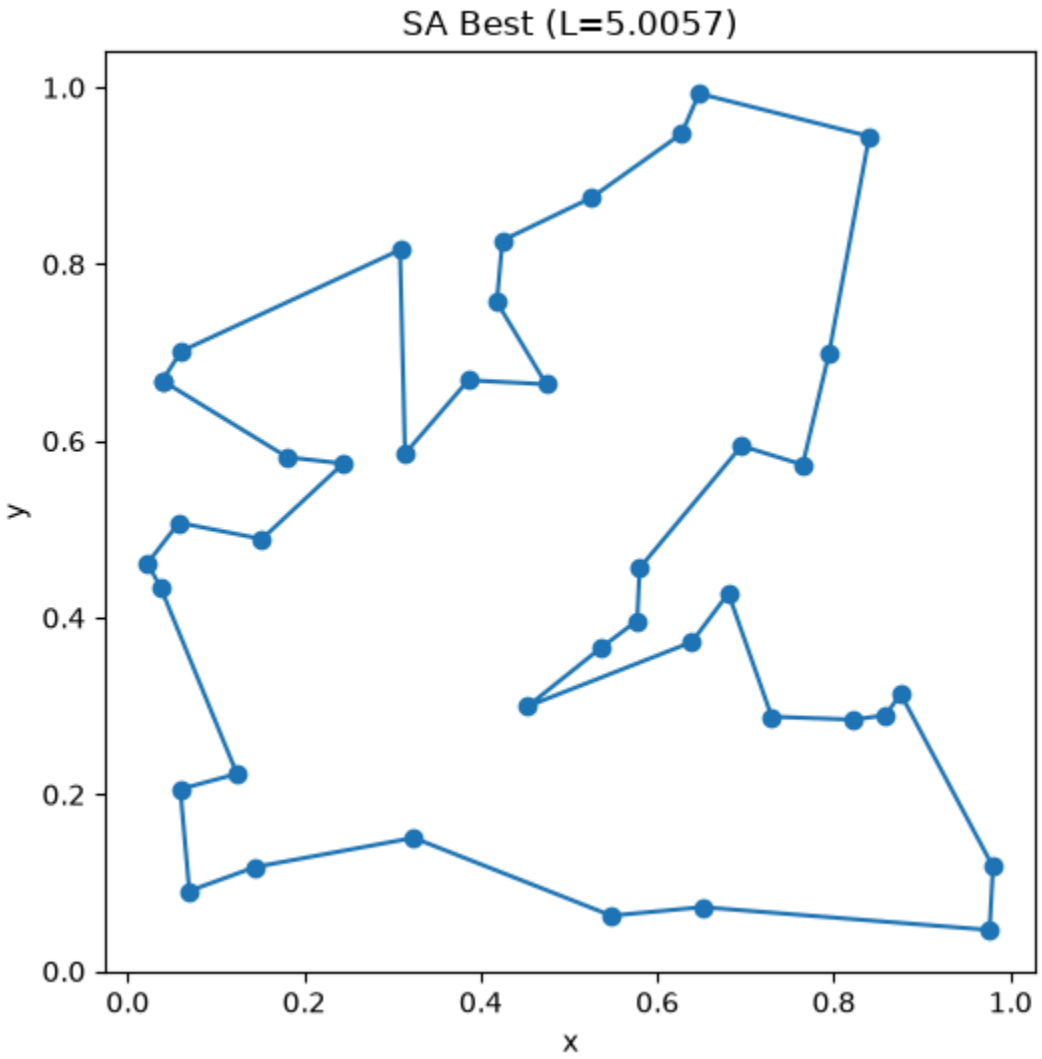
Abstract

The traveling salesman problem (TSP) asks for the shortest possible route that connects a list of cities exactly once and returns to the start, mimicking a tour taken by a traveling salesman as they make their rounds from city to city. In order to best optimize the paths taken, I utilized two different methods to decide which algorithm most efficiently creates a solution to the problem at hand. The two methods are Simulated Annealing (SA) and the Genetic Algorithm (GA). In simple terms, Simulated Annealing was able to create a more efficient solution than the Genetic Algorithm, though the Genetic Algorithm converged on a solution faster than Simulated Annealing.

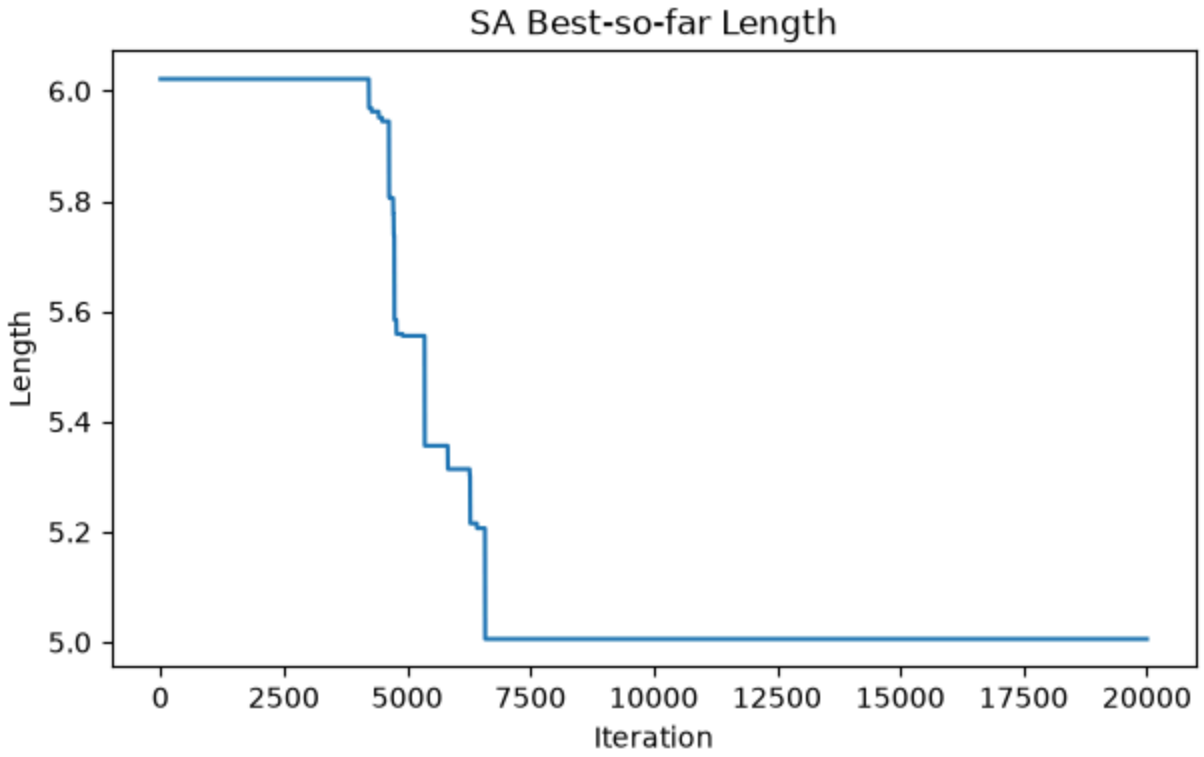
The two algorithms were compared against a baseline of a greedy nearest neighbor approach to solving the TSP, which found a shortest path of length 6.0214.



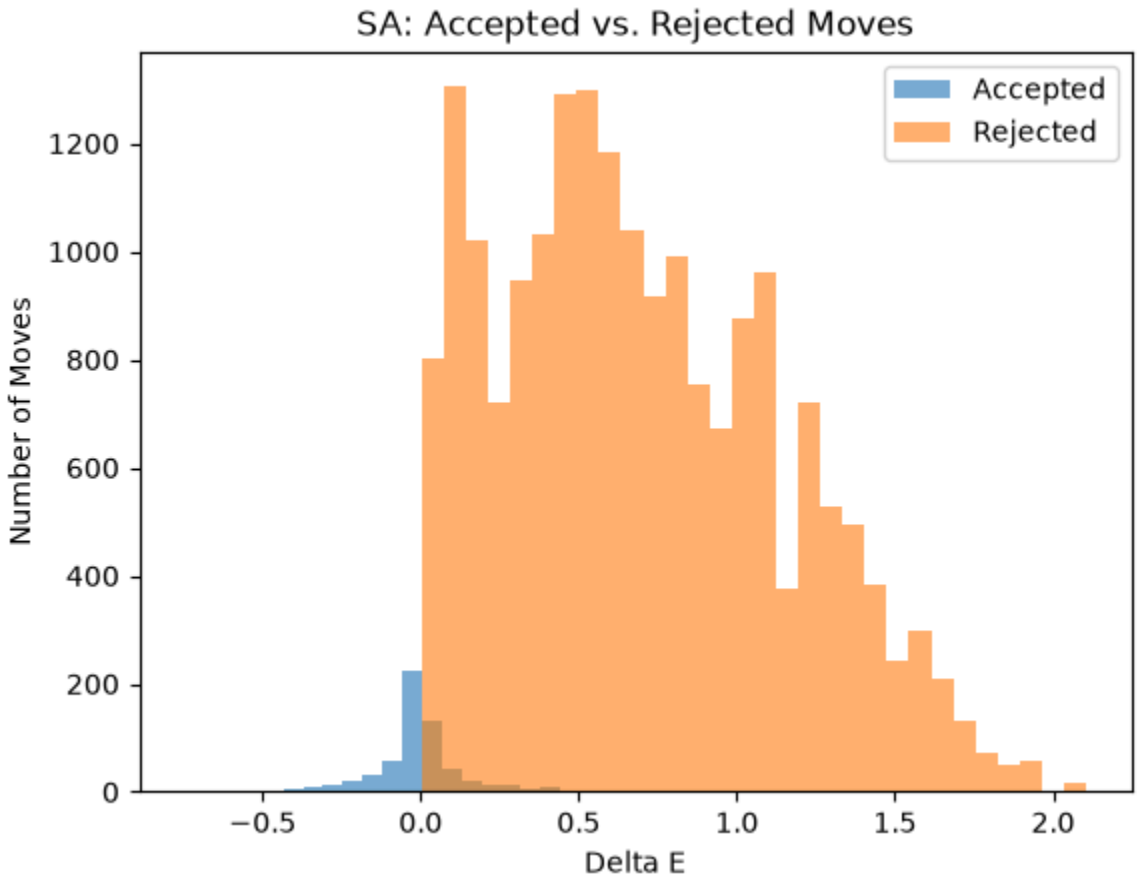
Simulated Annealing Results



Simulated Annealing converged to a best distance of 5.0057, beating the Nearest Neighbor greedy approach by being 16.87% shorter. Something to note about the results here is that it took nearly 7500 iterations to converge on its solution, however.


SA Best-so-far Length
Length
6.0
5.8
5.6
5.4
5.2
5.0
0
2500
5000
7500
10000
12500
15000
17500
20000
Iteration

Local Minima



As we can see in this histogram, SA took positive Delta E moves which showed evidence of it moving uphill. With that in mind, the best-so-far graph tends to show that the algorithm would always trend towards a better solution even if it had to take a subpar path at certain points.

Genetic Algorithm Solution

```python
def tournament_select(pop: List[Tour], lengths: List[float], k: int, rng: random.Random)
    # TODO: tournament selection (minimize length)
    competitors = rng.sample(range(len(pop)), k)
    winner = min(competitors, key =lambda i: lengths[i])
    return pop[winner]
    #return winner

def order_crossover_ox(parent1: Tour, parent2: Tour, rng: random.Random) -> Tour:
    # TODO: order crossover (OX)

    #decide cut indices
    cut_idx: List[int] = (rng.sample(range(len(parent1)), 2))
    cut_idx.sort()

    #create child list:
    child: List = []
    for x in range(len(parent1)):
        child.append(None)

    #take from parent 1
    for x in range(cut_idx[0], cut_idx[1]+1):
        child[x] = parent1[x]

    #now from parent 2
    start = cut_idx[1] + 1
    child_idx = (cut_idx[1] + 1) % len (child)
    for offset in range(len(parent2)):
        idx = (start + offset) % len(parent2)
        if parent2[idx] not in child:
            child[child_idx] = parent2[idx]
            child_idx = (child_idx+1) % len(child)
    return child

def mutate_swap(tour: Tour, rng: random.Random) -> Tour:
    # TODO: swap mutation
    idx1, idx2 = rng.sample(range(len(tour)), 2)
    tour[idx1], tour[idx2] = tour[idx2], tour[idx1]
    return tour
```

To implement the genetic algorithm I had to fill in three spots: tournament select, order crossover, and mutate swap.

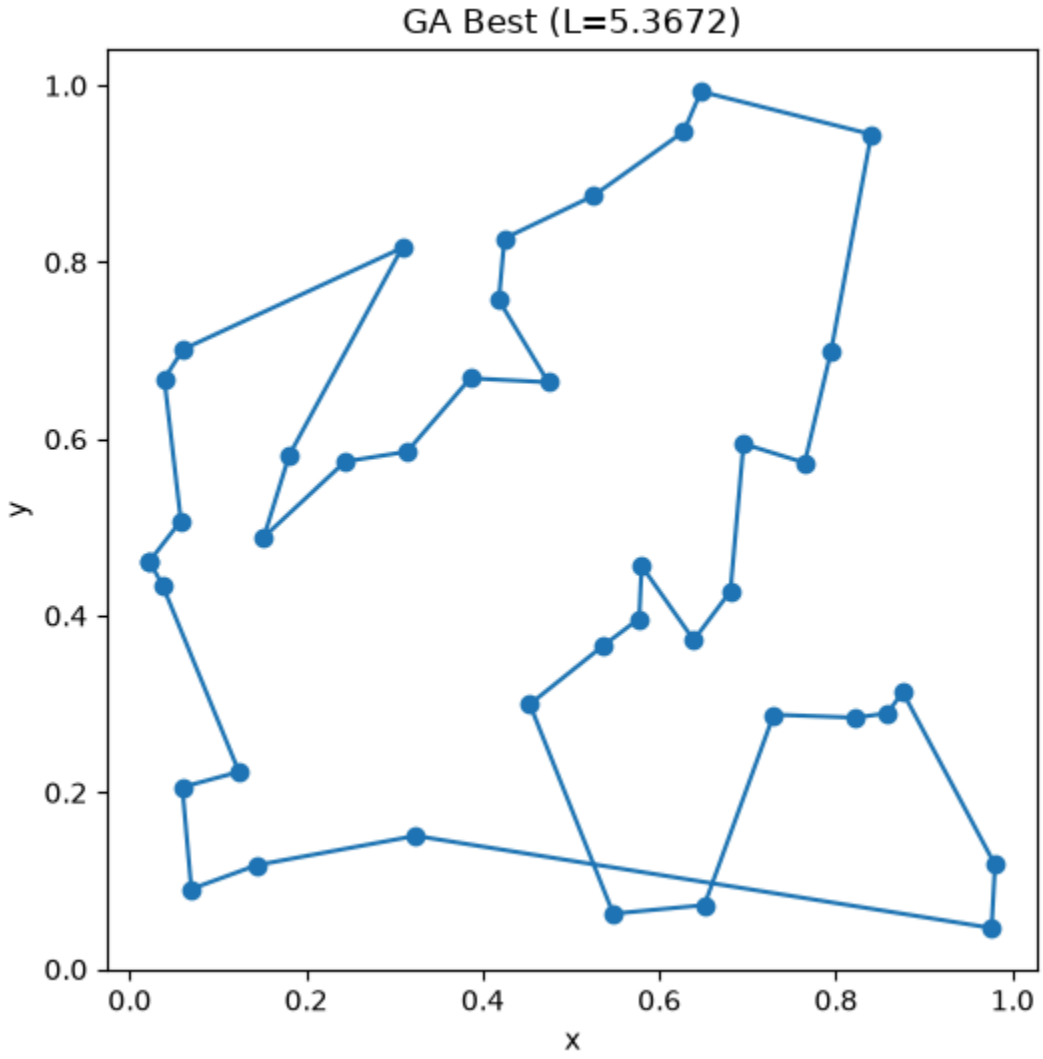
Tournament select was a matter of sampling a random group from the supplied population and selecting the best result from the group. Rather simple implementation, reflected by how few lines it took to implement.

Order crossover took the most time to implement due to myself having a difficult time figuring out how to get the iterator to wrap around to the front of a list in python. Eventually I found that using a modulo operator allowed for me to return to the front of a list upon exceeding the length, avoiding an out of range error and continuing to search for elements not taken during the first cut.

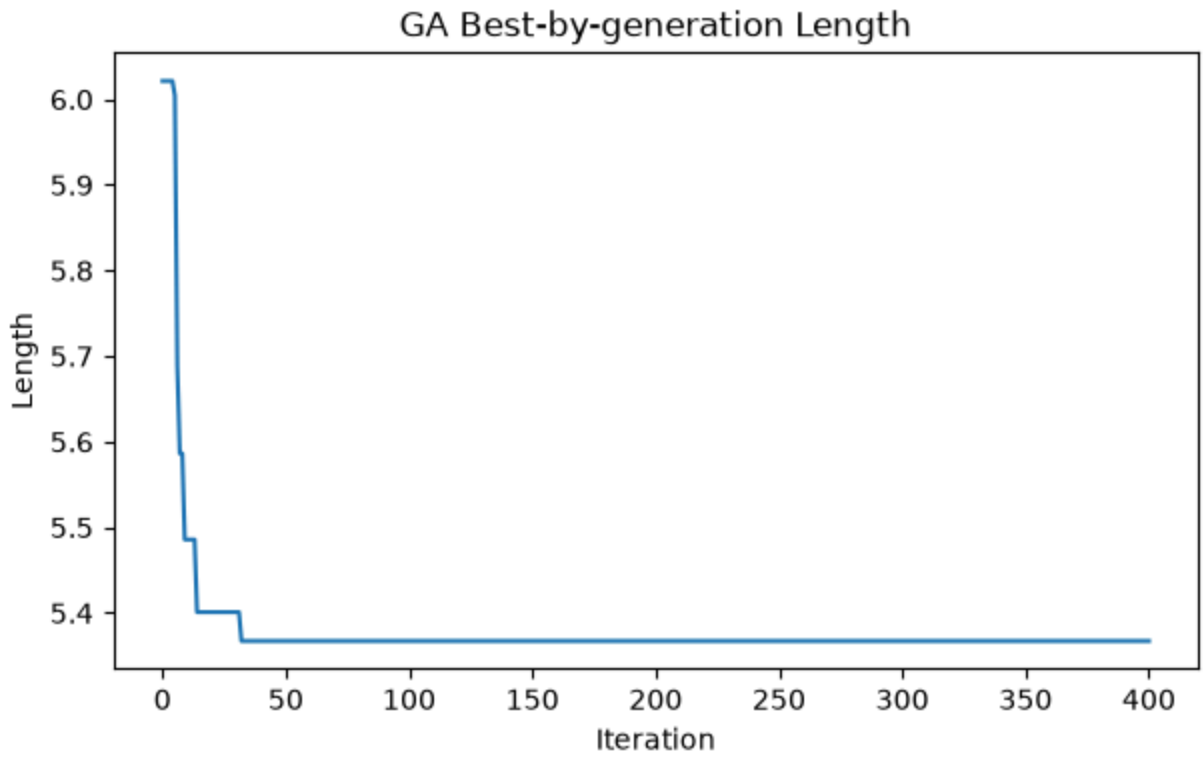
Similarly to tournament select, mutate swap was just a matter of taking two random indices and swapping them, which is something I'm used to doing thanks to learning sorting algorithms. This of course being different due to using random sampling instead of set swaps.

Genetic Algorithm Results

The Genetic Algorithm's improvement over Nearest Neighbor was only 10.86%, less than SA at a best distance of 5.3672.



Where the genetic algorithm outperformed SA was in how quickly it converged on this result, achieving convergence in less than 50 iterations when compared to the almost 7500 of Simulated annealing.


GA Best-by-generation Length
Length
6.0
5.9
5.8
5.7
5.6
5.5
5.4
0
50
100
150
200
250
300
350
400
Iteration